INF3132 – Programmation Orientée Objet
S5 2023–2024

esiea

Cours 1 :

La Programmation Orientée Objet
Les bases de Java

Felicia IONASCU
Siba HAIDAR
Aylen RICCA

Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

Les paradigmes de programmation

- Les paradigmes sont un moyen de classification des langages de programmation, basé sur leurs fonctionnalités
  - Ex : impérative, déclarative, parallèle, fonctionnelle, orientée objet …

- Les langages modernes les plus utilisés sont multi-paradigme : ils combinent plusieurs fonctionnalités

La programmation Déclarative

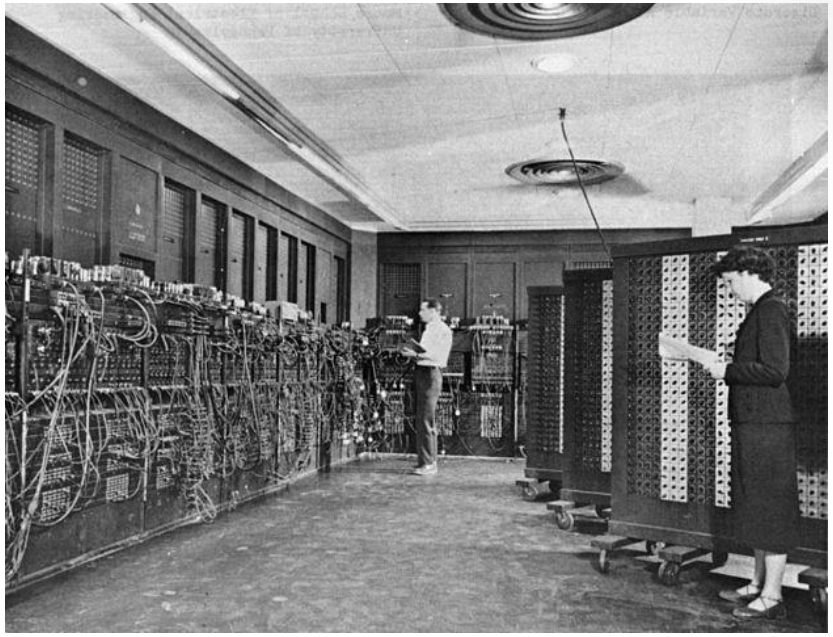
- Se concentre sur quoi exécuter
- Définit la logique du programme, mais ne définit pas le flux de contrôle
- Sous-types :
  - La programmation **fonctionnelle** : les programmes sont des évaluations de fonctions mathématiques; on évite les états et les données mutables
    - C++, Java (since version 8), Lisp, Python, R, Ruby, Scala, JavaScript, Elm, Wolfram Language, Matlab
  - La programmation en **flux de données** : les programmes sont une série de connexions entre les données; une opération s'exécute automatiquement quand toutes ses entrées sont disponibles
    - LabVIEW, Simulink, Tensorflow (API)

La programmation Impérative

- Se concentre sur comment exécuter le programme
- Définit le flux de contrôle en commandes qui changent l'état d'un programme
- Sous-types :
  - Programmation **procédurale**, programmation **structurée** :
    - spécifient quels sont les pas qu'un programme doit exécuter pour arriver dans un état désiré
    - C, C++, Lisp, PHP, Python, Matlab, Wolfram Language
  - Programmation **orientée objet**
    - organise les programmes en objets, qui sont des structures de données contenant des champs et des méthodes, avec leurs interactions
    - C++, C#, Java, Python

Historique de la POO (1/2)

- de la machine vers l'humain -



Programmation Machine — 1945: Plankalkül

- Modèles de bits, représentés en octal ou hexadécimal
- Chaque modèle agit sur les circuits du CPU pour exécuter une opération fondamentale sur le matériel

Programmation Assembleur — 1949: ShortCode

- Représente le langage machine sous une forme lisible par un humain
- Les combinaisons de bits du langage machine sont représentées par des «mnémoniques »
- Chaque instruction est une tache spécifique: charge, additionne, sauve, saute sur des donnés dans les registres
- Représentation bas-niveau d'un programme compilé,
- Code spécifique pour chaque CPU , très rapide

ARM, x86, Motorola, IBM, MIPS

Historique de la POO (2/2)

- de la machine vers l'humain -



Programmation Structurée — 1958: ALGOL

- Flux du programme structuré
- Selection (if/then/else), repetition (while, for)
- Structures bloc ({}), indentation
- Sous-routines

Imperatif

ALGOL, PL/I, ADA

Programmation Procedurale — 1960: Fortran

- Appel de procédure

Modulaire

Fortran, COBOL, BASIC, Pascal, C

Programmation Orientée Objet — 1967: Simula

- Basé sur le concept d'objet
- 4 principes: encapsulation, abstraction des données, polymorphisme, heritage

C++, C#, Eiffel, Java, PHP, Python, Ruby, Scala, JavaScript, Matlab

Quelques exemples

Langage	Paradigmes	Quand	Auteur	Entreprise
Assembleur				
Fortran	structuré, procédural, orienté objet	1957	J. Backus	IBM
Pascal	impératif, structuré	1970	N.Wirth	
C	impératif, structuré	1972	D.Ritchie	Bell Labs
Matlab	fonctionnel, impératif, procédural, orienté objet	fin '70		Mathworks
C++	procédural, fonctionnel, orienté objet	1985	B.Stroustrup	
Python	fonctionnel, impératif, procédural, orienté objet	1990	G. van Rossum	Python Software Foundation
Java	Impératif, orienté objet (orienté classe), fonctionnel	1995	J.Gosling	Sun Microsystems
Javascript	événementielle, fonctionnel, impératif, orienté objet	1995	B.Eich	Netscape
Julia	dispatch multiple, procédural, fonctionnel, meta	2012	Jeff Bezanson, Alan Edelman, Stefan Karpinski, Viral B. Shah	Open development

- Allez sur wooclap.com
  - Question 2 et 3



Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

La POO: Principes Directeurs

- Se **distancer** du matériel
  - langages de haut niveau

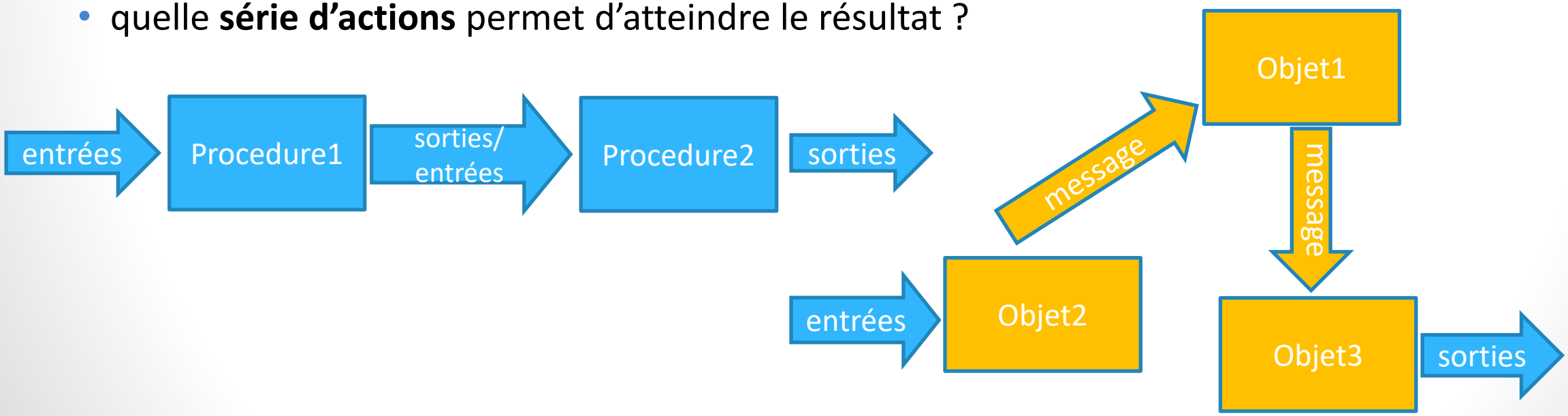
- Rapprocher la programmation informatique des **modes de pensée humains** de résolution de problèmes
  - En utilisant notamment des approches issues des sciences cognitives
  - L'humain perçoit le monde comme étant composé d'objets

- Les données (et non pas les instructions) sont au cœur du programme

L'approche Orientée Objet

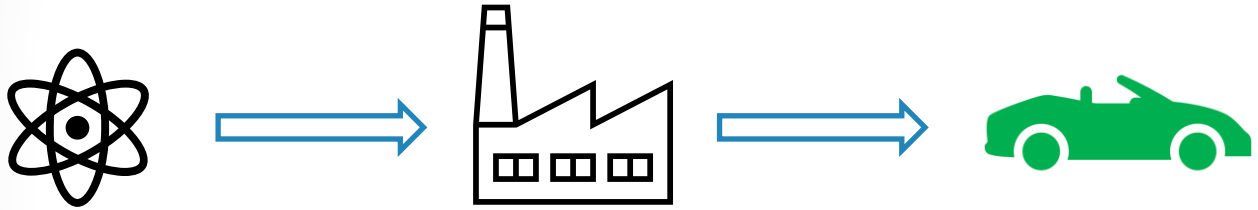
- L'approche Orientée Objet (OO) aborde les problèmes selon leurs **aspects structuraux**
  - qui sont les **acteurs** (objets), quels sont leurs **attributs** et comment **collaborent-ils** pour résoudre le problème ?

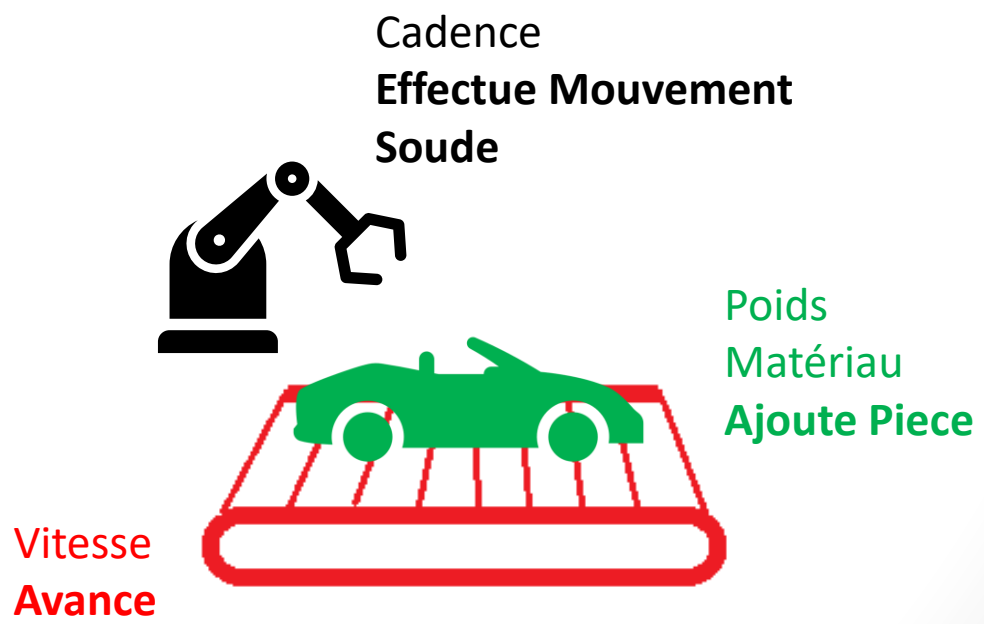
- En cela, elle **se distingue de l'approche procédurale** qui les aborde dans une perspective de traitement de l'information
  - quelle **série d'actions** permet d'atteindre le résultat ?



L'approche Orientée Objet

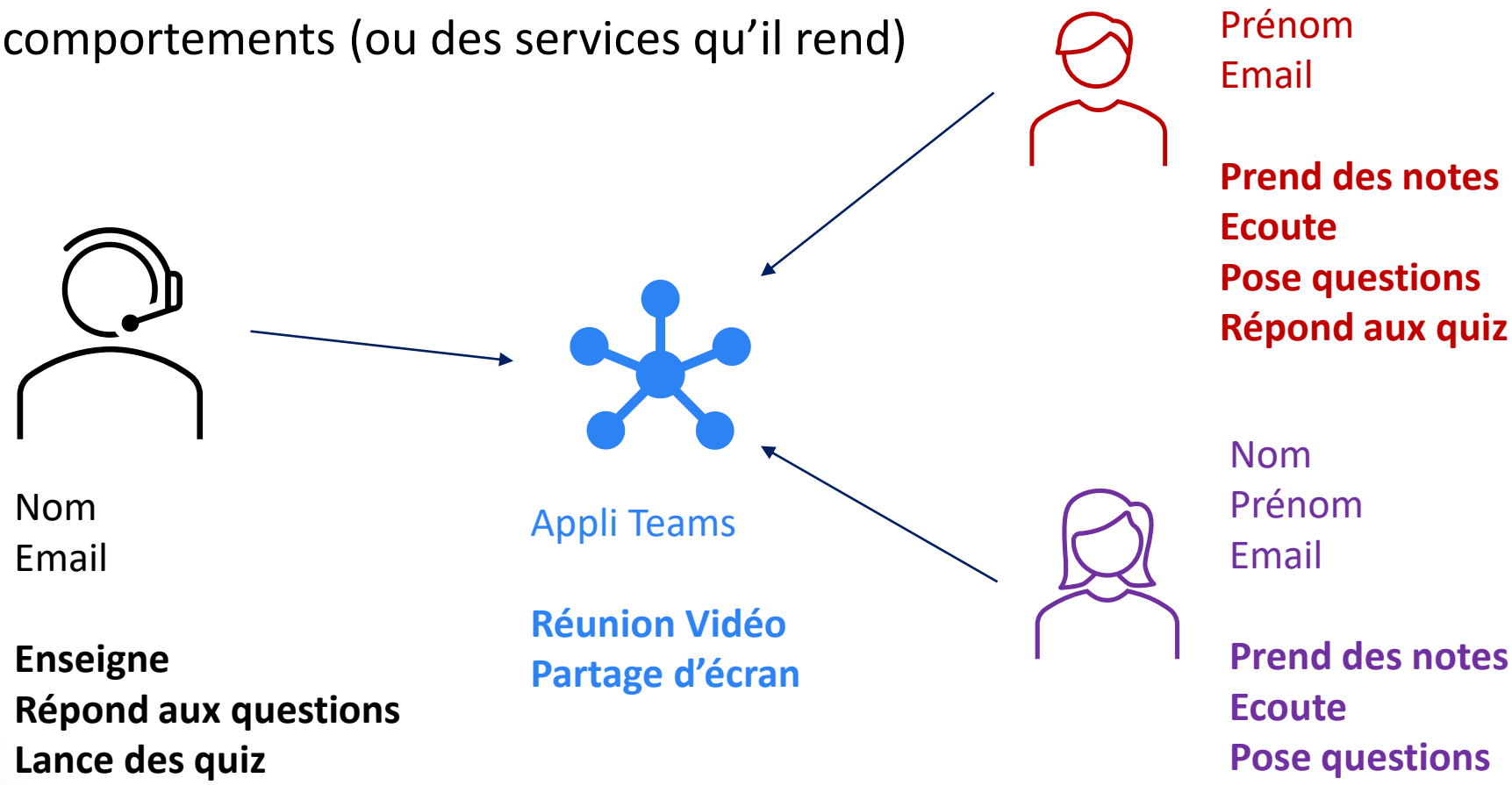
- Un objet est défini par :
  - Des attributs
  - Des comportements (ou des services qu'il rend)





L'approche Orientée Objet

- Un objet est défini par :
  - Des attributs
  - Des comportements (ou des services qu'il rend)



Approche Orientée Objet vs. Approche Procédurale

- En OO, les attributs et les comportements sont regroupés ensemble
- En Procédurale, les attributs et les comportements sont séparés

- L'OO est un **niveau d'abstraction** supplémentaire à l'approche procédurale

- Cependant, votre programme sera au final traduit en approche procédurale, qui donne des instructions à faire à la machine

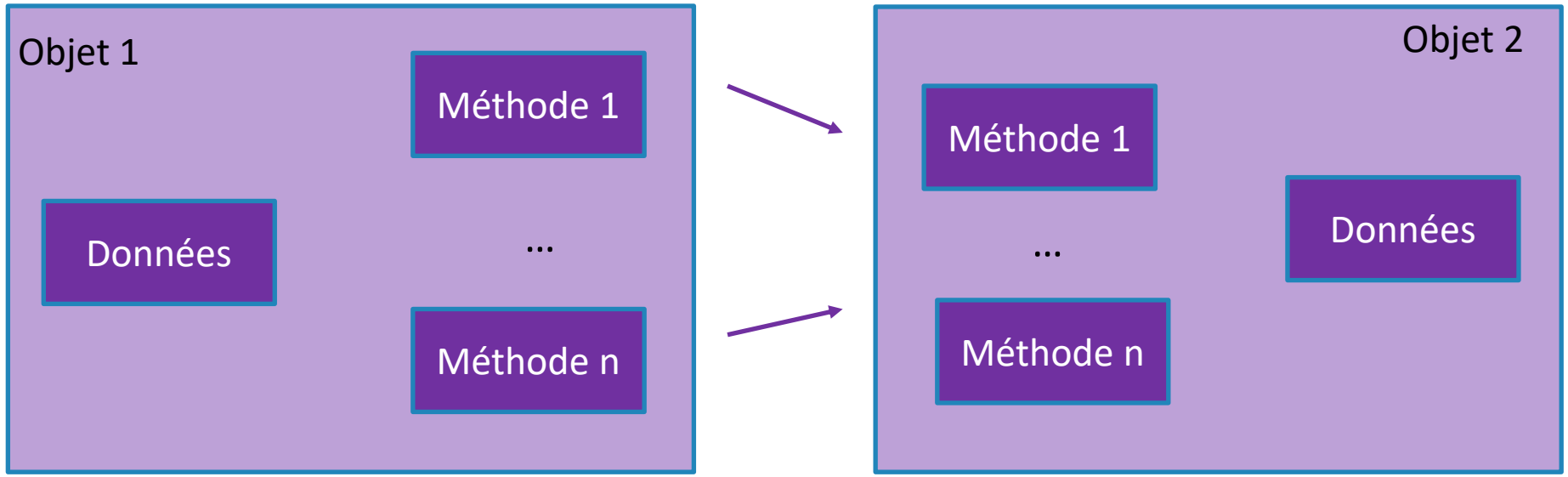
Approche Orientée Objet vs. Approche Procédurale

- Procédurale :



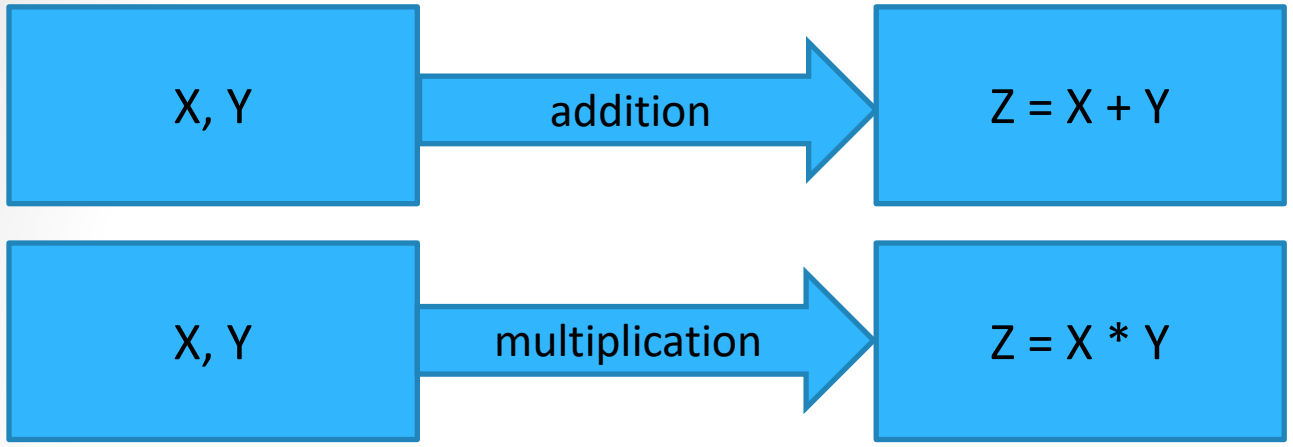
Les fonctions agissent sur des données globales

- Objet :



Les méthodes sont définies ensemble avec les données dans un objet

Exemple simple



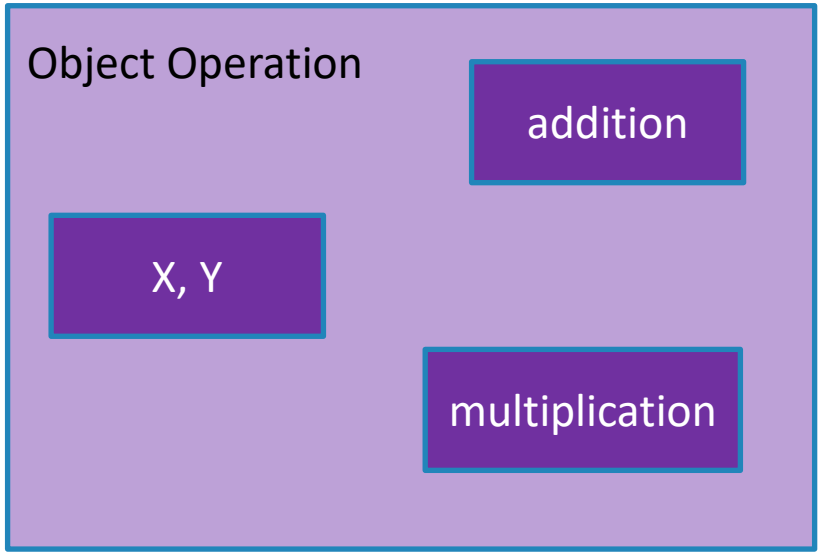
- Procédurale

moodle

Cours 1 - Question 1

- Écrivez ce programme en C

- Objet



Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

La perception (1/2)

esiea



La perception (2/2)

- Le cerveau **segmente** la scène en **objets pour s'y repérer**













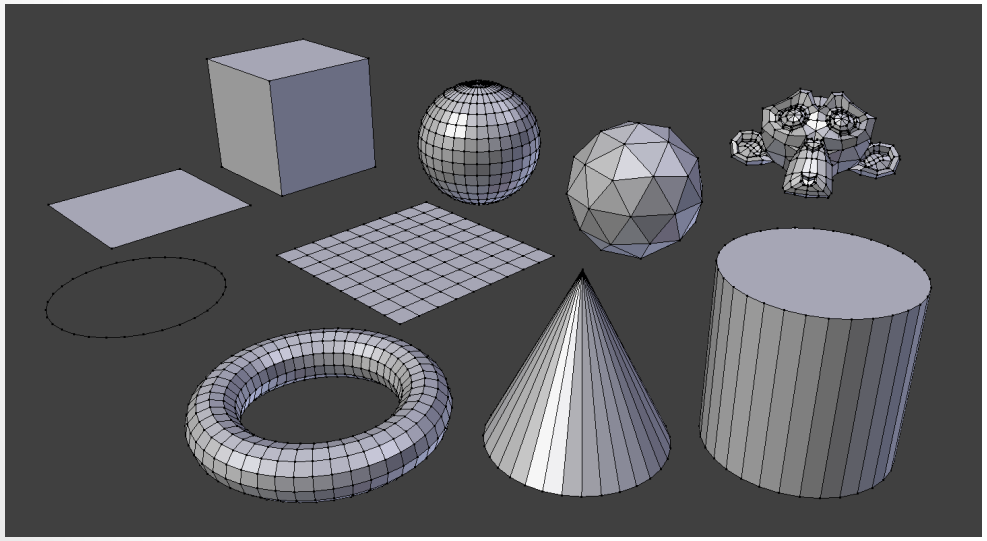
- Un objet est une **entité** (*bus, feu, immeuble*...) que l'on identifie grâce à :
  - l'ensemble de ses **attributs** (*forme, couleur, vitesse, âge, marque*...)
  - et de leurs **valeurs** particulières (*ronde, rouge, 35 km/h, 42 ans, Volvo*..)

Objets simples et objets complexes (1/2)











Objets simples et objets complexes (2/2)





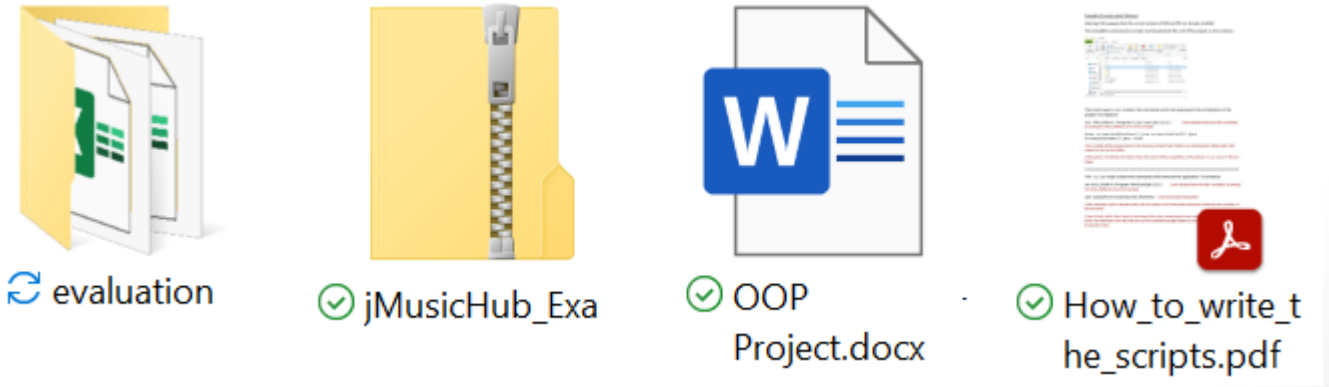
Les jardins de Bouges le Chateau

Exercice

- Allez sur wooclap.com
  - Question 4 et 5

- Identifiez les attributs qui caractérisent ces objets :

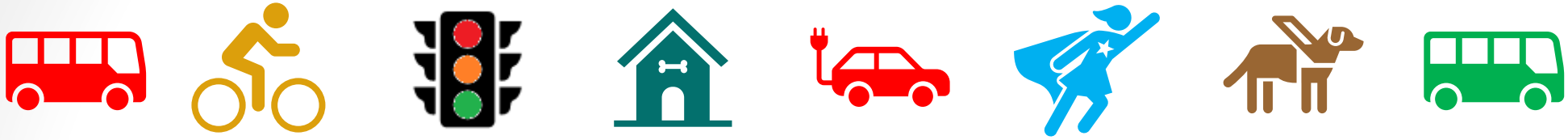




Plan

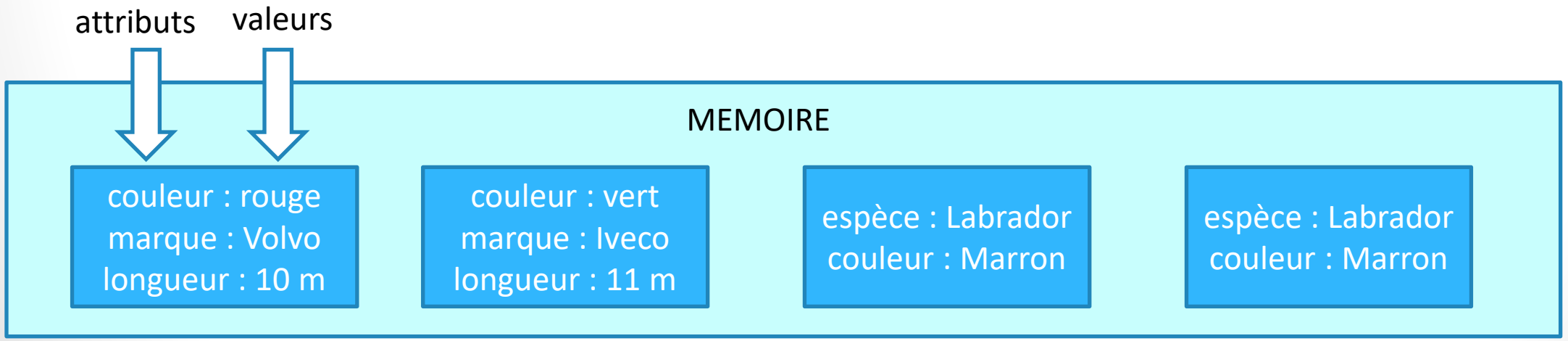
1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

Représentation en mémoire d'un objet (1/3)



- Un objet est une **entité** (*bus, voiture, passant, chien*. . .) que l'on identifie grâce à :
  - l'ensemble de ses **attributs** (*forme, couleur, vitesse, âge, marque*...)
  - et de leurs **valeurs** particulières (*carrée, rouge, 35 km/h, 22 ans, Volvo*...)

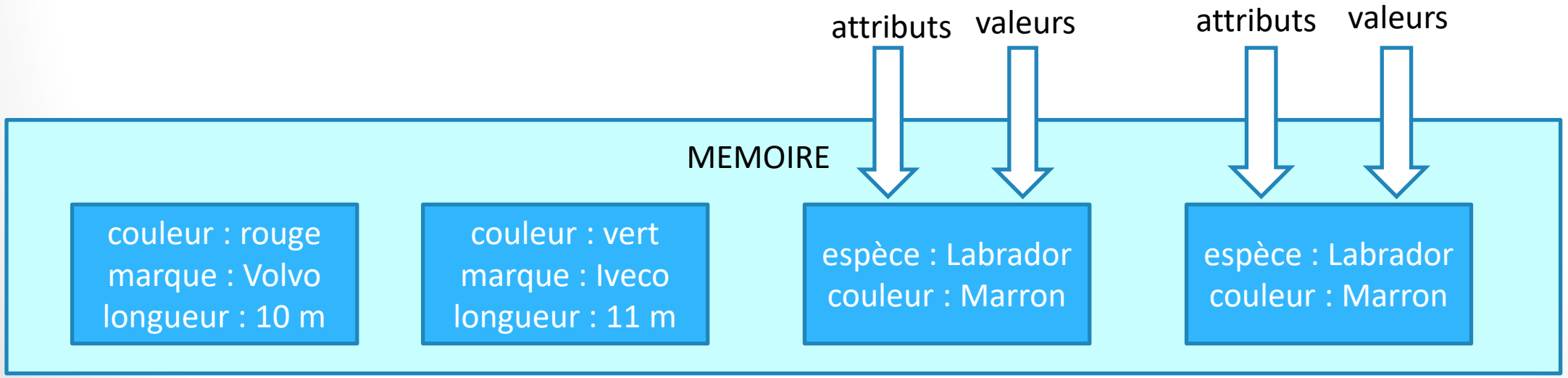
- En OO, on stocke et on manipule ces objets en mémoire comme des **ensembles de couples** attributs/valeurs



Représentation en mémoire d'un objet (2/3)

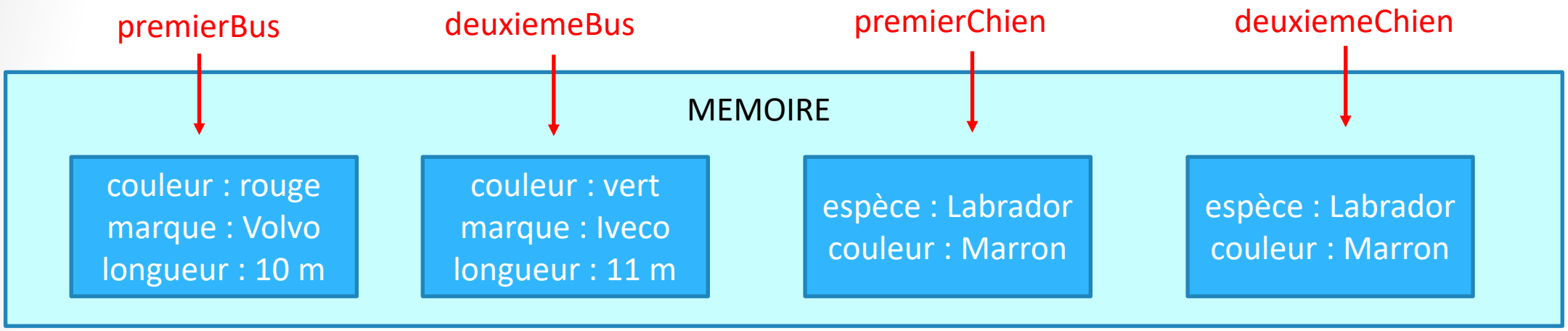
- Chaque objet possède son propre espace mémoire
  - Certains objets ont les mêmes attributs mais pas les mêmes valeurs
  - Certains objets ont les mêmes attributs et les mêmes valeurs

Comment savoir de quel objet on parle ?



Représentation en mémoire d'un objet (3/3)

- Pour utiliser un objet, il faut pouvoir le désigner : c'est le **REFERENT**

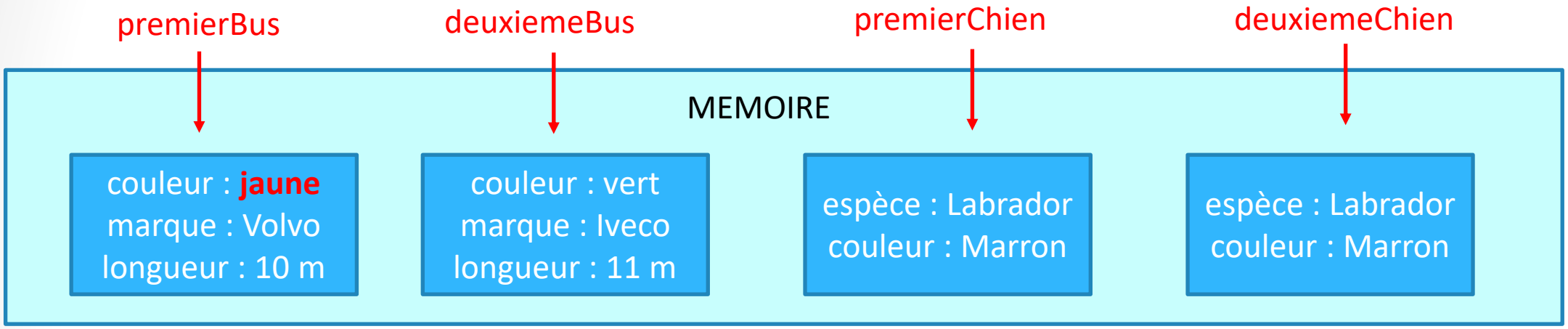


- Un référent est **une variable** qui permet d'**accéder aux attributs d'un objet** via un **opérateur**
- En Java, l'opérateur est le **. (*point*)**

```
premierBus.couleur
```

Propriétés des référents (1/2)

- Le **référent** permet d'accéder aux attributs d'un objet **en lecture** comme **en écriture**

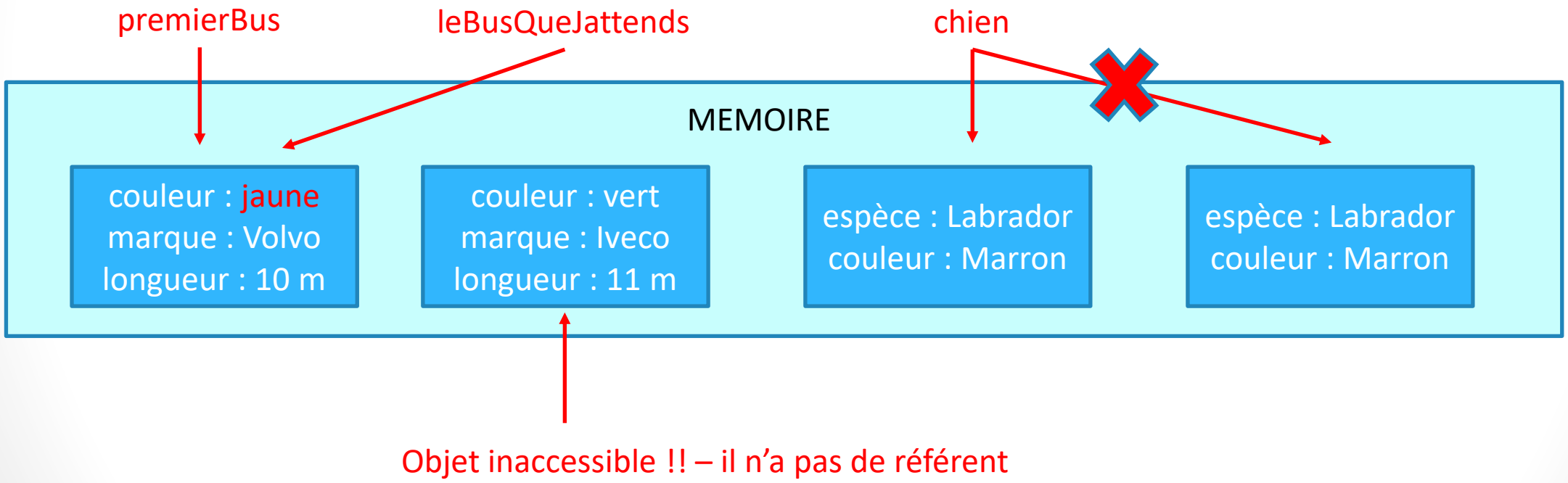



```
si (premierBus.longueur > 10 m)
      afficher « attention, bus trop long »

premierBus.couleur = jaune
```

Propriétés des référents (2/2)

- On peut avoir de **multiples référents** sur un même objet
- En revanche, un référent ne peut **désigner qu'un seul objet**
- Un objet qui n'a **plus de référent** devient **inaccessible**



Référent et pointeur

- Au final, les référents ressemblent beaucoup aux **pointeurs**
- Rappel : en C/C++, les pointeurs sont des **variables** permettant de **stocker des adresses** mémoires et par ce biais d'**accéder aux variables** dans la mémoire via l'**opérateur * (*étoile*)**

Référent	**Pointeur**
- désigne un objet	- contient une adresse
- accès "direct" `maVoiture`	- accès via l'opérateur * `(*maVoiture)`
- accès aux attributs via l'opérateur . `maVoiture.couleur`	- accès aux attributs via l'opérateur . `(*maVoiture).couleur`
- pas d'arithmétique ~~`(maVoiture+1).couleur`~~	- arithmétique `(*(maVoiture+1)).couleur`

Exercice



Cours 1 - Question 2

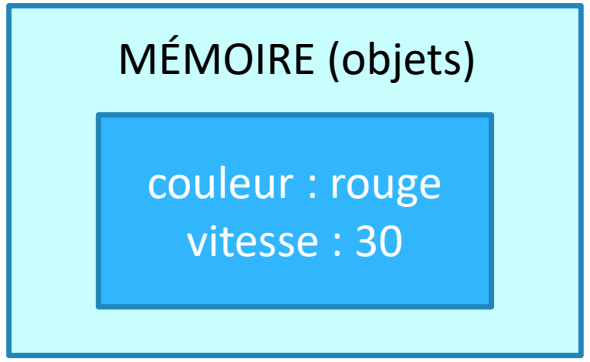
- Ecrivez en C une fonction `changeTrafficLightColor`
- Quelle garantie avons-nous que la fonction est appelée uniquement pour un feu tricolore ?

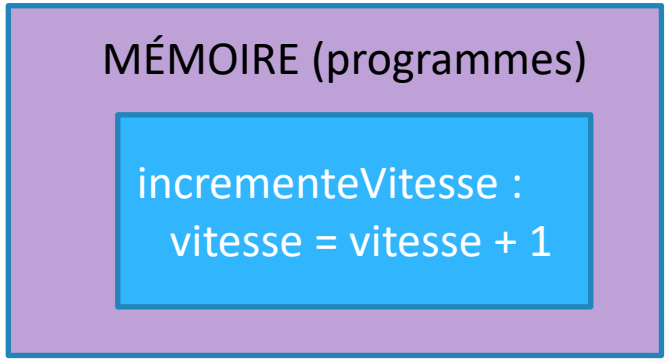
Évolution des objets (1/2)

Une scène observée n'est pas figée !

- Le feu tricolore change de couleur, les voitures s'arrêtent, les passants traversent, ...
- Les objets conservent leur identité mais **changent d'état** : ils doivent pouvoir évoluer et permettre d'interagir avec eux
- Un changement d'état implique une modification de la valeur des attributs de l'objet, donc une modification de la zone mémoire associée
- Comment procéder à ces changements d'états ?
  - Via des fonctions stockées dans la mémoire des programmes :







Approche procédurale

- Dans l'approche procédurale (en C par exemple), les **données** sont considérées comme des **entrées** ou des **sorties**

```
int incrementeVitesse(int val) {
    val += 10;
    return val;
}
```

```
int vitesse = 0;
vitesse = incrementeVitesse(vitesse);
```

```
int age = 0;
age = incrementeVitesse(age);
```

- **Problème** : le compilateur ne vérifie que la concordance de **type !**
- => On peut incrémenter n'importe quelle variable de type `int` (pas uniquement des vitesses !!)

Approche Objet : la classe

- Il n'y a pas de sens à vouloir appliquer la fonction `incrementeVitesse()` sur une autre entité qu'une vitesse

  => cela peut mener à des fonctionnements inattendus

  => un certain fonctionnement est propre à certaines entités

- En approche Objet, **les attributs** d'une entité et les fonctions (que nous appellerons désormais **méthodes**) sont regroupés au sein d'une même entité:

la classe

Exercice



Cours 1 - Question 3

- Ecrivez en Java une méthode `changeTrafficLightColor`
- Quelle garantie avons-nous que la fonction est appelée uniquement pour un feu tricolore ?

Approche Objet : les méthodes (1/5)

- Voici la classe `FeuTricolore` :

```
class FeuTricolore {
        // Attributs
        int couleur = 0;
        // Methodes
        void changeCouleurFeu() {
                couleur = couleur + 1;
                if ((couleur > 2)
                        couleur = 0;
  }
}
```

- Comme `couleur` et `changeCouleurFeu()` sont **dans la même classe** il n'est pas nécessaire de passer l'attribut en **argument** de la méthode
- **Les méthodes** ont automatiquement **accès** à tous les **attributs de leurs classes**

Approche Objet : les méthodes (2/5)

- Les méthodes peuvent avoir plusieurs arguments de types différents et un type de retour

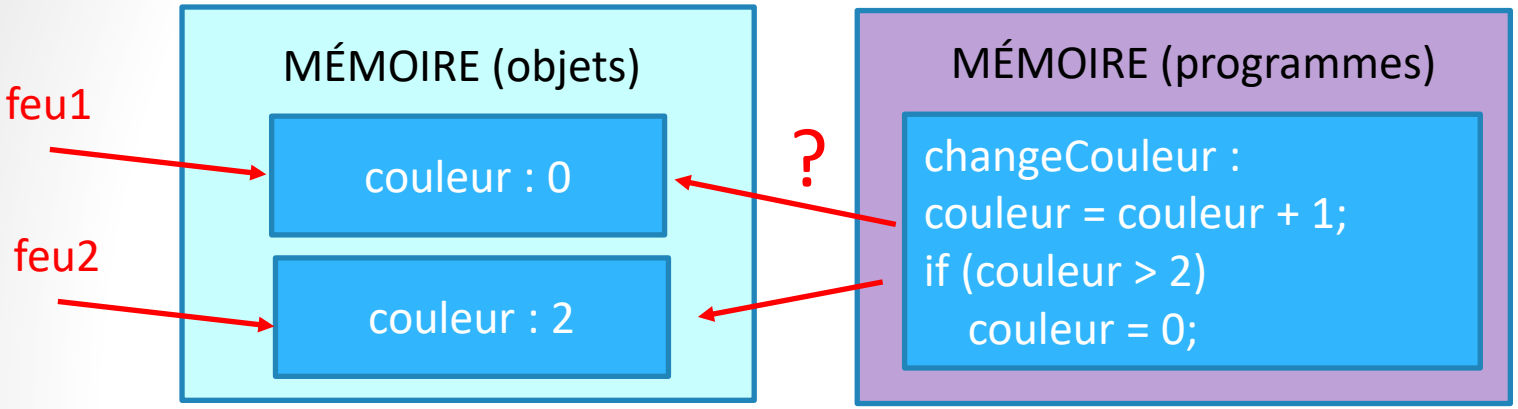
```
class FeuTricolore {
        // Attributs
        int couleur;
        // Méthodes
        // donne une nouvelle valeur à la couleur du feu
        void fixeCouleurFeu(int maCouleur) {
                couleur = maCouleur;
                if ((couleur > 2) || (couleur < 0))
                        couleur = 0;
        }
        //récupère la couleur actuelle du feu
        int quelleCouleurFeu() {
                return couleur;
        }
}
```

Approche Objet : les méthodes (3/5)

- On utilise **l'opérateur . (point)** pour accéder aux méthodes, comme pour les attributs :

```
feu.changeCouleurFeu();          //OK si feu est de type FeuTricolore
voiture.changeCouleurFeu();      //erreur de compilation
                                 //en supposant que voiture
                                 //n'est pas de type FeuTricolore
```

Approche Objet : les méthodes (4/5)



- Et si j'ai plusieurs objets ?
- Comment est-ce que la méthode sait de quel attribut je parle ?

- À nouveau, c'est l'opérateur **. (point)** qui va faire la différence

```
feu1.changeCouleurFeu();      //modifie l'attribut couleur de feu1
feu2.changeCouleurFeu();      //modifie l'attribut couleur de feu2
```

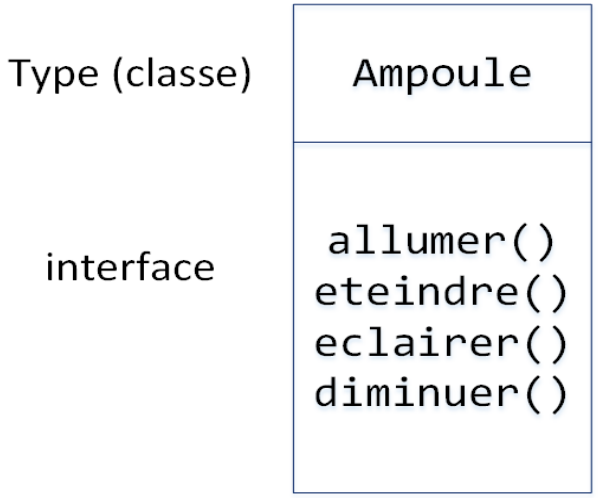
Approche Objet : les méthodes (5/5)

- En OO, on demande aux entités de faire des choses :
  changer sa couleur, se déplacer, répondre à une question, etc…
  - On appelle cela de l'**envoi de message**

- Les entités répondent uniquement aux messages qu'elles connaissent :
  elles **offrent des services**

Approche Objet : la classe et son interface (1/2)

- La **classe** décrit un ensemble d'objets qui ont des caractéristiques identiques (les attributs) et des comportements identiques (les méthodes)
- Une **classe** est un **type** de donnée
- L'ensemble des requêtes qu'on peut faire aux objets d'une classe (les méthodes) constituent son **interface**
- L'interface est un contrat avec les clients de la classe (d'autres classes qui l'utilisent)
- Le code qui satisfait la requête est **l'implémentation** de l'interface



Récapitulatif : Classe, Attributs et Méthodes

- Une **classe** définit un **type** d'entité : un feu tricolore, une voiture, un humain
- Elle définit donc leurs **attributs** (caractéristiques, propriétés)
  - ex : *couleur, kilométrage, âge*
- et les actions possibles qu'on peut leur demander (les **méthodes**)
  - ex : *changeCouleur, accelere, demandeNom*
- La classe **lie** les attributs et les méthodes ensemble dans une entité **qui a du sens**
  - *On ne demande pas d'écouter à un feu tricolore, mais on peut lui demander de changer sa couleur*
- Les méthodes accessibles d'une classe constituent **l'interface** de la classe

Instance de classe

- La **classe** définit un **type** d'entité :

  ⇒ nos objets sont donc des **instances** (des réalisations) de ces classes

  ⇒ on peut instancier **autant d'objets que nécessaire** pour résoudre notre problème

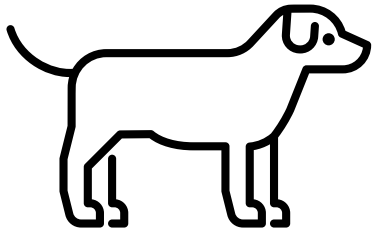
- D'un point de vue programmation, **instancier** un objet, c'est lui **allouer** un **espace mémoire** et **spécifier des valeurs** pour ses attributs

  => Exemple de classe :

  - Chien (attributs : nom, âge, race)

  => Exemple d'instances :

  - MonChien (Lassie, 4 ans, Collet)
  - LeChienDuVoisin (Belle, 3 ans, Chien des Pyrénées)







http://www.publicdomainpictures.net/view-image.php?image=59364&picture=dog-collien-portratt

Constructeur

- En Java, l'**instanciation** se fait **via l'opérateur new et le nom de la classe**

```
monChien = new Chien();
```

- En général, le constructeur a des **arguments qui permettent d'initialiser les attributs** du nouvel objet

```
monChien = new Chien("Lassie", Collet);
```

- La variable qui reçoit l'objet doit être un **référent** de **même type**

```
Chien monChien = new Chien("Lassie", Collet);        //OK!
Chat  monChat = new Chien("Minou", Birman);          //erreur
```

Exemple de classe avec constructeur

```
class FeuTricolore {
        // Attributs
        int couleur;
        // Constructeurs
        FeuTricolore () {
            couleur = 0;
        }
        FeuTricolore (int c) {
            couleur = c;
        }
        // Methodes
        void changeCouleurFeu() {
            couleur = couleur + 1;
            if (couleur > 2)
                couleur = 0;
        }
}
```

Exercice



Cours 1 - Question 4

- Rajoutez des constructeurs à votre classe `FeuTricolore`

Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

Les langages purs orientés objet

- Sous-ensemble des langages orientés objet
- 7 caractéristiques :
  1. **Encapsulation** – les classes regroupent des attributs et des méthodes
  2. **Héritage** – une nouvelle classe peut inclure les caractéristiques d'une autre classe
  3. **Polymorphisme** – fournir une interface unique à des entités pouvant avoir différents types
  4. **Abstraction** – mécanisme qui permet de créer un type générique regroupant les caractéristiques communes des entités
  5. **Tous les types prédéfinis sont des objets**
  6. **Tous les types définis par l'utilisateur sont des objets**
  7. **Toutes** les opérations sur les objets doivent s'effectuer uniquement par des **méthodes accessibles des objets**
- Example : *Smalltalk*
- Java n'est pas un langage **pur** orienté objet :
  - Java a des types primitifs (`int`, `double`, `char`)
  - Les opérations sur les objets de classe `Integer`, `Double`, etc., se font en utilisant les types primitifs (`int`, `double`)

Hello World : Le fichier principal

- En Java, **chaque fichier source contient la définition d'une unique classe** (i.e., un type / un concept / une entité)

- Par ailleurs, **rien n'existe en dehors des classes**, il va donc nous falloir commencer par une **classe principale**

  (`HelloWorld` par ex.) qui contiendra la méthode ... principale :

  la méthode `main`

- Cela nous donne le fichier `HelloWorld.java` :

```
class HelloWorld {
        public static void main(String args[])  {
                /* ... */
        }
}
```

Hello World : Décryptage

```
class HelloWorld { /* ... */ }
```

- On **déclare** une nouvelle classe nommée `HelloWorld`
- Dans cet exemple, la classe n'a pas d'attribut et de méthode en propre. Elle ne représente pas vraiment une entité mais sert juste à contenir la méthode `main`

```
public static void main(String args[])  { /*...*/ }
```

- La méthode `main` sert de **point d'entrée** au programme
  - C'est la méthode qui est appelée lors de l'**exécution**.
- Nous verrons plus tard ce que signifient `public`, `static` et `String`
  - mais retenez que la méthode `main` doit respecter exactement cette syntaxe

Hello World : Interaction console

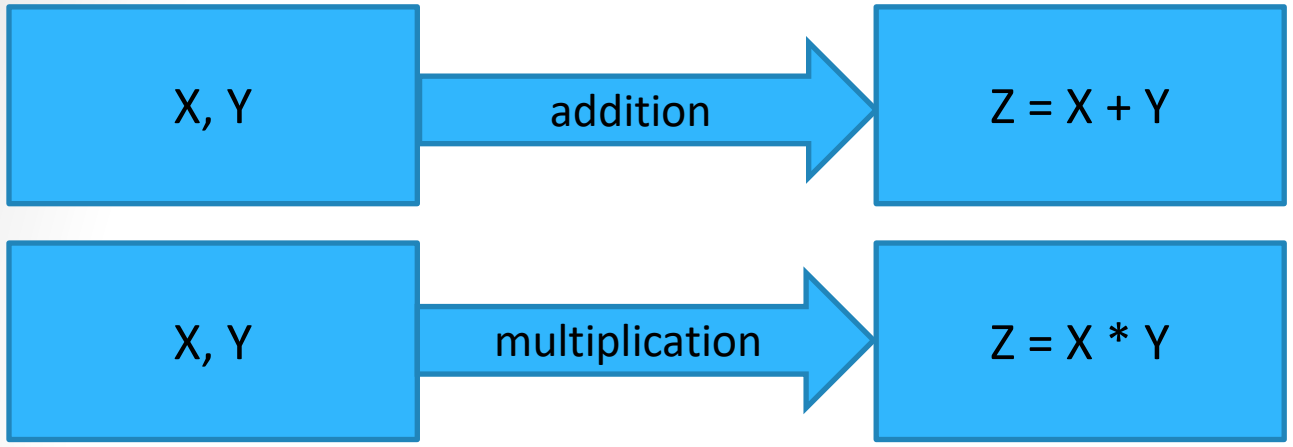
- En Java, lorsqu'on veut interagir avec la console (soit pour afficher du texte, soit pour récupérer une saisie utilisateur), on utilise la classe `System`
- La classe `System` possède trois attributs :
  - **`out`** : sortie standard
  - **`err`** : flux d'erreur
  - **`in`** : les données entrantes
- Ces attributs sont des **objets**, ils possèdent donc des méthodes permettant d'interagir avec eux
- La méthode `println` permet d'envoyer des messages textes sur les flux de sortie (`out` et `err`)

```
System.out.println("message à la console");
System.err.println("erreur !");
```

Hello World : Code final

```
class HelloWorld {
	public static void main(String args[])  {
		System.out.println ("Hello World !");
	}
}
```

Exemple simple

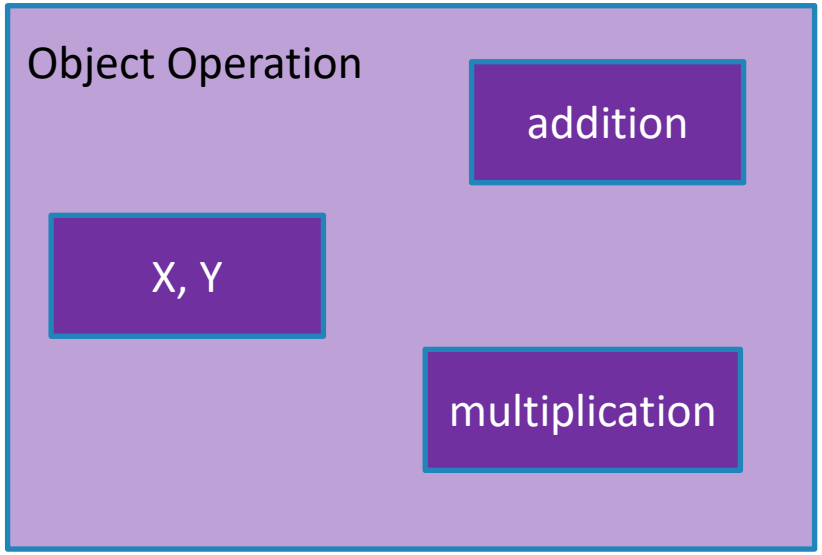


- Procédurale

Cours 1 - Question 5

- Écrivez ce programme en Java

- Objet



Les composants de Java

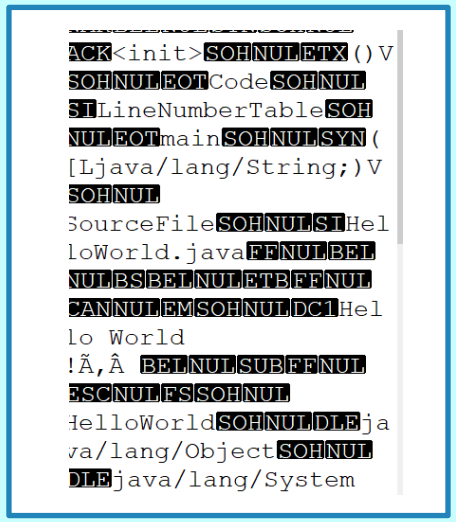
- **JDK**: **J**ava **D**evelopment **K**it
  - outils de développement (e.g., le compilateur *javac*, le désassembleur *javap*, le débuggeur *jdb*, java archive *jar*)
  - *JRE* (pour exécuter les programmes) *privé* = JRE de production + bibliothèques utiles aux développeurs (e.g., internationalisation)
- **JRE**: **J**ava **R**untime **E**ngine
  - outils d'exécution de programmes Java
  - contient : technos de déploiement, toolkits IHM, bibliothèques
  - JVM
- **JVM**: **J**ava **V**irtual **M**achine
  - permet aux programmes Java de s'exécuter sur n'importe quel matériel ou système d'exploitation (*portabilité*)
  - gère et optimise la mémoire du programme

Hello World : Compiler et exécuter

1. **Compilation** – javac (JDK)

```
class HelloWorld {
public static void
main(String
args[])  {
System.out.println
("Hello World
! ");
}
```

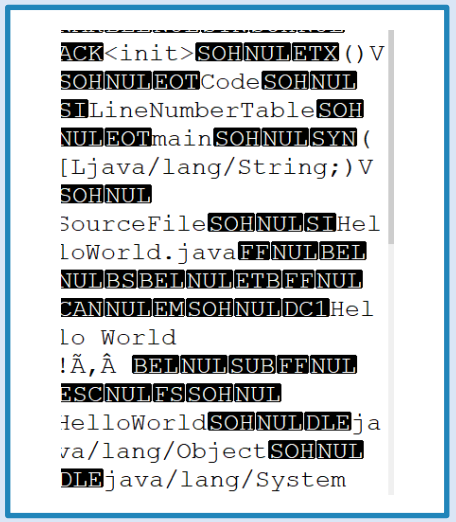
Fichiers sources



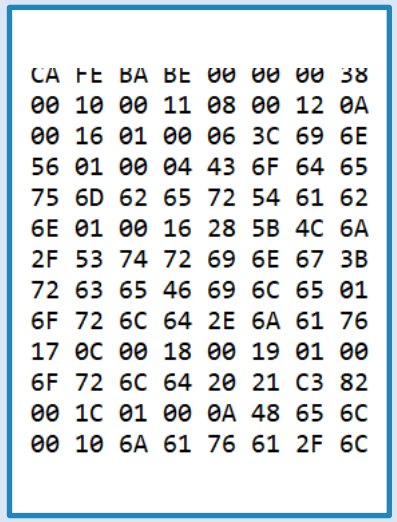
Bytecode portable

2. **Exécution** – java (JRE) interpréteur
Compilation à la volée

JVM



Bytecode portable



Code machine

```
C:\> javac HelloWorld.java
C:\> java HelloWorld
C:\> Hello World !
```

Variables et identificateurs

```
public class ClasseAvecIdentificateur {
    int id = 0;                         //variable d'instance
    static boolean isIdentifier;        //variable de classe

    void doSomething(String parameter) {   //paramètre (argument)
         Object currentValue = null;       //variable locale
    }
}
```

- Identificateurs = les noms des variables
  - Caractères : A à Z et a à z
  - Chiffres : 0 à 9. Les chiffres ne peuvent pas se placer en début du nom de variable
  - Caractères spéciaux : uniquement $ (dollar) et _ (*underscore*, clé de soulignement)
    Mais pas en début de l'identificateur :

    `$currentValue` n'est pas permis
    `currentValue$` est permis

Types primitifs et types Wrapper

- Il existe des types de données primitives qui sont définies de manière identique **sur toutes les plateformes**

Type primitif	Taille	Min	Max	Type wrapper	
boolean	-	-	-	Boolean	true / false
char	16-bit	Unicode 0	Unicode 2^{16}- 1	Character	
byte	8-bit	-128	+127	Byte	
short	16-bit	-2^{15}	$+2^{15}$-1	Short	
int	32-bit	-2^{31}	$+2^{31}$—1	Integer	
long	64-bit	-2^{63}	$+2^{63}$—1	Long	
float	32-bit	IEEE754	IEEE754	Float	
double	64-bit	IEEE754	IEEE754	Double	
void	-	-	-	Void	
				BigInteger	N’importe quelle grandeur, mais précision arbitraire
				BigDecimal	

Types primitifs

- Pour les réels à virgule flottante :
  - En fonction de la langue du système (FR / EN) c'est soit la virgule '**,**' soit le point '**.**' qui sert de séparateur des décimales dans la console / dans les fichiers
  - Dans le code, c'est toujours le point '**.**' qui est utilisé

```
$ LC_ALL=fr_FR.UTF-8 java Main
Valeur du double : 5,6
```

```
$ LC_ALL=en_US.UTF-8 java Main
Valeur du double : 5.6
```

Opérateurs

- Java possède les mêmes opérateurs de base que le C :
  - Calcul : +, -, *, /, %,
  - Affectation : =,
  - Incrémentation : ++, −−, +=, -=, *=, /=, %=,
  - Test : ==, !=, <=, <, >, >=,
  - Conjonction logique : &&, ||, !

- Java n'a pas de **`sizeof`**`()`, car tous les types primitifs ont la même taille en octets sur toutes les machines
- Astuce : pour éviter de mémoriser la précédence des opérateurs et pour rendre le code lisible, on utilise des parenthèses

Embranchement

```
if (test) {
        // do something
} else {
        //do something else
}
```

```
switch (day) {
        case 1: System.out.println("Monday");
                break;
        case 2: System.out.println("Tuesday");
                break;
        default: System.out.println("Invalid month");
                break;
}
```

```
System.out.println (switch (day) {
        case 1 -> "Monday";
        case 2 -> "Tuesday";
        default -> throw new IllegalStateException ("Invalid day" + day);
        }
);
```

Depuis Java SE 12

Boucles (1/2)

```
while (test) {
        //do something
}
```

```
do {
        //do something
} while (test);
```

```
for (int i = 0; i < 100; i++) {
        //do something
}
```

- Remarque : `i` a une portée locale et n'est valable que dans la boucle `for`

Boucles (2/2)

- La boucle « Pour chaque » : *foreach*

```
int intArray[] = {1, 2, 4, 8, 16, 32, 64, 128};
for (int v : intArray) {
        System.out.println (v);
}
```

- À savoir :
  - On peut sortir d'une boucle avec le mot clef **break**
  - On peut sauter la fin du tour avec le mot clef **continue**
  - On peut sortir de la méthode avec le mot clef **return**
  - (même si le type de retour est **void**)

Literals

- Les **Literals** sont des séquences de caractères (chiffres, lettres et autres) qui représentent des valeurs constantes à stocker dans des variables
- `F` et `f` = `float`, `D` et `d` = `double`, `L` et `l` = `long` → désambiguation

```
class Literals {
    long carteCredit = 1234_5678_9012_3456L; //séparateur _ pour lisibilité
    byte b = 0x7f;           //valeur max byte hex
    short s = 0x7fff;        //valeur max short hex
    int i1 = 0x2f;           //hexadécimal (lowercase)
    int i2 = 0x2F;           //hexadécimal (uppercase)
    int i3 = 0177;           //octal (préfix zéro)
                             //Hex et Oct marchent avec long aussi
    long n1 = 200L;          //long suffix
    //! long n4(200);        //interdit
    float f1 = 1;
    float f2 = 1F, f3 = 1f, f4 = 1e-45f, f5 = 1e+9f; //float suffix
    double d1 = 1d, d2 = 1D, d3 = 47e47d;            //double suffix
}
```

Les méthodes

- Rappel : Les méthodes implémentent les comportements des objets
- Définition générale de méthode :

```
MyClass {
      ...
      public ReturnType methodName(ParamOneType parameter1, ParamTwoType parameter2) {
            ...
            return returnVariable;
      }
      ...
}
```

  - Type de retour : `ReturnType` ou `void` si la méthode ne retourne rien
  - Nom : `methodName`
  - Types des paramètres : `ParamOneType`, `ParamTwoType`
  - Noms des paramètres : `parameter1`, `parameter2`

Interaction console : via les classes System et Scanner

- Afficher du texte dans la console :

```
System.out.println ("introduisez un mot:");
System.err.println ("erreur!");
System.out.printf  ("valeurNumerique: %.3f", v);
```

- Récupérer du texte au clavier (`java.util.Scanner`) :

```
Scanner scan = new Scanner (System.in);
int i = scan.nextInt();
```

Exercice



Cours 1 - Question 6

- Class Pixel

String

- En Java, les chaînes de caractères sont des objets (et non pas des tableaux de caractères !)

```
String myName = new String("Ionascu")                    //constructeur
String mySchool = "ESIEA";                               //allocation par initialisation

//opérateur + : concaténation
double height = 1.60;
String sentence = "je mesure "  + height + "cm";

//accesseur sur la longueur de la chaine
int longueur = sentence.length();

//est-ce que la chaine contient la sous-chaine "cm" ?
boolean cm = sentence.contains("cm");
```

Types primitifs et classes Wrapper

- Une classe Wrapper encapsule des types primitifs, afin d'en créer des objets
- Utilité des Wrappers :
  - Dans les collections (`ArrayList`, `Vector`) on peut stocker uniquement des objets
  - Pour la synchronisation en multithreading
  - Pour des conversions de type

- Les Wrappers sont :
  - **Immuables** (on ne peut pas changer leur valeur)
  - `final` (on ne peut pas en hériter)

```
int i = 3;
Integer I = new Integer(i);
long l = I.longValue();
```

Tableaux

- En Java, les tableaux sont des objets

```
int tab1[];                    //références sur des tableaux
int[] tab2;

tab1 = new int[5];             //allocation dynamique :
                               //  Pixel[] tab = new Pixel[8];
int[] tab2 = {1, 2, 3};        //allocation par initialisation

tab1[0] = 3;                   //affectation de la première case du tableau
nbVal = tab2.length;           //attribut public final des tableaux

tab2[6] = 3                    //ArrayIndexOutOfBoundException
for (Pixel val: tab1) {
    val.changeColor(255, 0, 0);
    System.out.println (val);
}      //iterator
```

Exercice



Cours 1 - Question 7

- Tableau de pixels

Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling language (UML)

Le compilateur Java

- Syntaxe :

```
javac [options] [fichiers] [@fichiers]
```

- au moins un nom de fichier contenant du code source Java
- si plusieurs fichiers, on les précise un par un, séparés par des caractères espace ou en utilisant les jokers du système d'exploitation

  l'extension `.java` est obligatoire dans les noms des fichiers-source et doit être précisée sur la ligne de commande

```
javac ClasseTest.java
javac MonRepertoire\*.java
javac @listeDeFichiersJava
```

L'interpréteur Java (1/2)

- Interpréteur de bytecode :
  - lance le JRE, charge les classes nécessaires
  - exécute la méthode main de la classe passée en paramètre
  - ouvre une console pour recevoir les messages de l'application
- Note : utiliser `javaw.exe` pour ne pas ouvrir de console
- Syntaxe :

```
java [ options ] classe [ argument ... ]
java [ options ] -jar fichier.jar [ argument ... ]
```

Le paramètre *`classe`* = nom du fichier `.class` qui contient le **`main`** dont il ne faut pas préciser l'extension

Le **`.jar`** est une archive contenant des fichiers `.class` (créée avec l'utilitaire **`jar.exe`**)

L'interpréteur Java (2/2)

```
java [ options ] classe [ argument ... ]
```

- On peut indiquer des chemins et/ou des archives `.jar` de recherche des classes nécessaires à l'exécution avec l'option `-classpath`
- Cette option annule l'utilisation de la variable système `CLASSPATH`

- Exemple :

```
cd d:\Projects
REM HelloWorld.java est compilé et HelloWorld.class est créé dans le répertoire MyFirst :
javac MyFirst\HelloWorld.java
REM l'interpréteur cherche le .class dans le répertoire MyFirst
java -classpath d:\Projects\MyFirst HelloWorld
```

Génération automatique de documentation : javaDoc (1/3)

- L'écriture de commentaires dans le code est le meilleur moyen de création et maintien de la documentation
- Les commentaires sont de deux types :
  - **Commentaires d'implémentation** – pour que les développeurs comprennent le code

```
/* ... mon commentaire ... */ ou    // mon commentaire
```

  - **Commentaires de documentation** – pour créer la documentation du logiciel (sa spécification)

```
/** commentaire de documentation */
```

```
/** Un commentaire de documentation de classe */
public class docTest {
        // Un commentaire d'implémentation de variable
        public int i;
        /* Un commentaire de documentation de méthode */
        public void f() {}
}
```

Génération automatique de documentation : javaDoc (2/3)

- Javadoc génère un document HTML à partir des **commentaires de documentation** du code :

  https://www.oracle.com/technetwork/java/javase/tech/index-137868.html

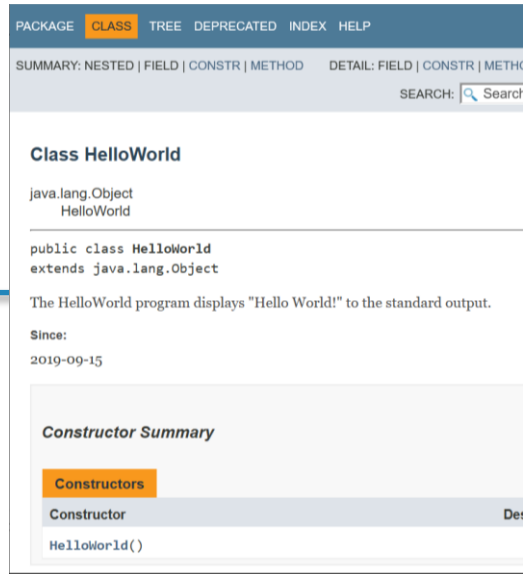
```
/** Un commentaire de documentation de classe */
public class docTest {
        /** Un commentaire de documentation de variable */
        public int i;
        /** Un commentaire de documentation de methode */
        public void f() {}
}
```

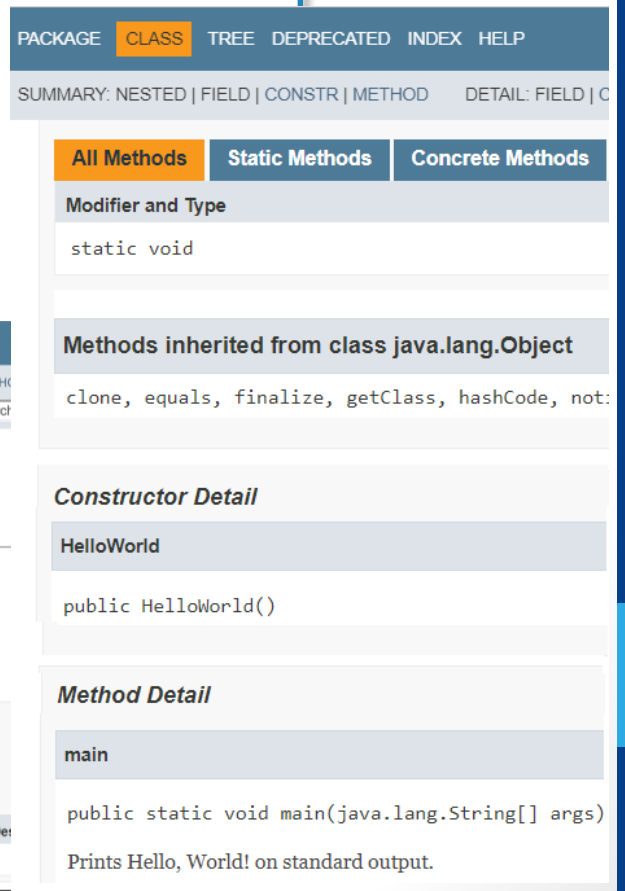
- appel en ligne de commande :

```
javadoc    docTest.java
```

Génération automatique de documentation : javaDoc (3/3)

```java
/**
* The HelloWorld program displays "Hello World!" to the standard output.
*
* @author  Felicia Ionascu
* @version 1.0
* @since   2019-09-15
*/
public class HelloWorld {
/** Prints Hello, World! on standard output.*/
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```





Les conventions de codage (1/3)

- L'importance des conventions de codage
  - Le logiciel est censé passer une grande partie de sa vie en maintenance
  - Les développeurs qui écrivent le logiciel sont rarement ceux qui le maintiennent
  - En utilisant des conventions de codage, on comprend le code plus rapidement, la lisibilité est améliorée

  - La version originale Oracle (1997)
    - https://www.oracle.com/technetwork/java/codeconventions-150003.pdf
  - Et les conventions publiées par Google:
    - https://google.github.io/styleguide/javaguide.html

Les conventions de codage (2/3)

- Quelques conventions importantes
  - Pas de fichier de plus de 2000 lignes
  - Chaque fichier contient une seule classe publique ou une seule interface publique
  - Les déclarations de variable se font en début de bloc { }
  - Initialiser les variables au moment de leur déclaration (sauf si l'initialisation se fait via un calcul)

```
class Sample extends Object {
      int ivar1;

      Sample(int i) {
            ivar1 = i;
      }

      int emptyMethod() {}
      ...
}
```

Accolade ouvrante sur la même ligne que la déclaration de classe ou méthode

Alignement

Pas d'espace entre un nom de méthode et la parenthèse qui ouvre sa liste de paramètres

Les conventions de codage (3/3)

- Quelques conventions importantes
  - Une seule déclaration par ligne `i++; j++;` interdit
  - Règles de Nommage

Type	Règle	Exemples

Classe	Utiliser des noms, commencer par majuscule, dans un nom de classe composé chaque mot commence par une majuscule. Pas d'acronyme ou abréviation (sauf si consacré ex. HTML)	`Lecteur`
		`LecteurDVD`
		`VideoRecepteur`

Interface	Idem classe	
Méthodes	Utiliser des verbes, commencer par minuscule; dans un nom de méthode composé, chaque mot interne commence par une majuscule	`setName()` `getEmail()`

Variables	Commencer par minuscule; dans un nom de variable composé, chaque mot internet commence par une majuscule
	Eviter d'utiliser les noms composés d'un seul charactère, à l'exception des variables « à jeter », dans des boucles par exemple
		`int i;`
		`float myWidth;`
		`LecteurDVD lecteurDVD;`

Constantes	Tout en majuscule, avec des « _ » pour séparer les mots	`int MIN_WIDTH = 4;`
		`int MAX_WIDTH = 999;`
		`int GET_THE_CPU = 1;`

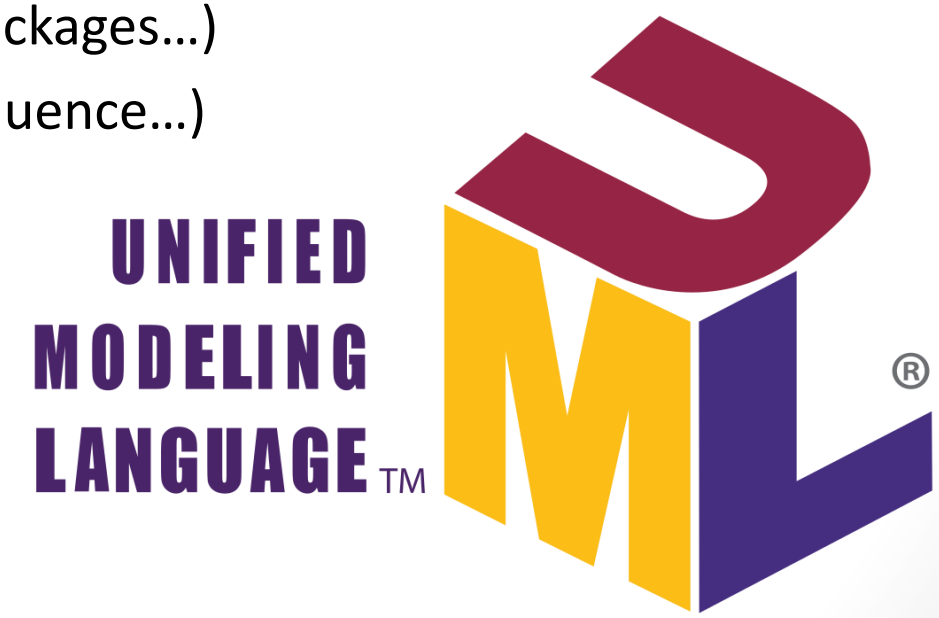
Plan

1. Les paradigmes de programmation
2. La POO: principes directeurs
3. Segmentation du monde
4. Le concept de Classe
5. Bases de Java
6. L'environnement Java
7. Le Unified Modeling Language (UML)

UML (Unified Modeling Language) (1/2)

- Format graphique normalisé de représentation des concepts objet
- Grady Booch, James Rumbaugh et Ivar Jacobson
- Pour la communication entre les développeurs d'un projet logiciel
- Version actuelle 2.5.1 (2017)

- Permet la description statique et dynamique d'un programme
  - Statique = la structure (**classes**, déploiement, packages…)
  - Dynamique = les interactions (collaboration, séquence…)



UML (Unified Modeling Language) (2/2)

- Représentation des attributs et des méthodes dans les diagrammes de classe

UML

Attributs

+ : public
- : private
: protected
$: static
{const}: final

A
+ att0: int + att1: int = 1 - att2: int = 1 # att3: int = 1 + CONSTANT: int = 12 {const} - array: int [20] - $ st: int = 1

Java

```
class A {
  public int att0;
  public int att1 = 1;
  private int att2 = 1;
  protected int att3 = 1;
  public static final int CONSTANT = 12;
  private int array[]  = new int[20]
  private static int st = 1
}
```

Methodes

A
+ m1 (p: int): int - m2 (p: int): int # m3 (p: int): int + m4 (p: int): int throws E; + m5 (p: int) int {abstract} + $ m6 (p: int): int

```
class A {
  public int m1 (int p) {...}
  public int m2 (int p) {...}
  protected int m3 (int p) {...}
  protected int m4 (int p) throws E;
  public abstract int m5 (int p);
  public static int m6 (int p) {...}
}
```

Exercice – Devoir (1/2)

- Avec la dernière JDK
  - https://www.oracle.com/java/technologies/downloads/
  - Lien « JDK Download »

- Récupérez le code de la classe `Pixel` dans un fichier nommé `Pixel.java`, sur votre disque en local
- Vérifiez si les conventions de nommage sont respectées
- Compilez la classe avec `javac`
- Exécutez le programme avec `java`
- Ecrivez les commentaires de documentation
- Générez la documentation avec `javadoc`
- Dessinez le diagramme *UML* de la classe

Exercice – Devoir (2/2)



TD 1 – Exercices de base

- Devoir individuel et en autonomie
- A faire avant la prochaine séance


esiea

Les plateformes Java

- **Java Card** – enables secure elements, such as smart cards and other tamper-resistant security chips, to host applications that employ Java technology
- **Java Platform, Micro Edition (Java ME)** – targeting environments with limited resources; embedded and mobile devices on the Internet of Things: micro-controllers, sensors, gateways, mobile phones, personal digital assistants (PDAs), TV set-top boxes, printers
- **Java Platform, Standard Edition (Java SE)** – targeting workstation environments: desktops and servers
- **Java Platform, Enterprise Edition (Java EE)** – targeting large distributed enterprise or Internet environments
- **Java TV (Java ME-based)** – targeting TV and set-top box devices to create Electronic Program Guides (EPG's), Video-on-Demand (VOD) clients, games and educational applications, applications for accessing internet data and, on most Blu-ray Disc titles, the user interface and bonus content